



Benha University
Faculty of Engineering
Electronics and Communication Engineering Department



Design and Implementation of a 5-Stage Pipelined MIPS Processor

Microarchitectural Specifications, Hazard Mitigation, and RTL Verification

Course: Computer Organization

Prepared by Team:

Ahmed Hisham Abdelfattah Mohamed
Mahmoud Farouk Ibrahim
Omar Khaled Ibrahim
Mahmoud Abdelwahab Mahmoud
Ali Haitham Habib

Submitted to:

Dr. Hatem Zakaria

May 25, 2026

Contents

1	Introduction	2
1.1	Project Scope and Toolchain Paradigm	2
1.2	Architectural Optimization Strategy	2
1.3	Microarchitectural Challenges and Core Framework	2
1.4	Verification and Deliverable Overview	2
2	5-Stage Pipeline Architecture	2
2.1	Microarchitectural Stage Description	3
2.2	Microarchitectural Layout Schematic	3
2.3	Interstage Boundary Isolation Registers	4
2.4	RTL Viewer Hardware Schematic Generation	4
3	Hazard Mitigation and Control Logic	4
3.1	Data Dependencies and the Operand Forwarding Unit	4
3.2	Load-Use Data Hazards and Pipeline Stalling	6
3.3	Control Hazards (Branch Flushes)	6
4	Hardware Component Implementation	6
4.1	Program Counter Register (PC)	6
4.2	Instruction Memory Storage Block	7
4.3	Multi-Port Unified Register File	7
4.4	Arithmetic Logic Unit (ALU)	8
4.5	ALU Local Decoding Logic Controller	9
4.6	Synchronous Byte-Addressable Data Memory	9
4.7	Central Control Logic Array Unit	10
4.8	Top Intertwined Integrated Control Subsystem	11
4.9	Core Pipeline Structural Datapath Unit	11
4.10	Top-Level MIPS Enclosure Wrapper	15
5	Verification and Simulation Results	16
5.1	Verification Firmware	16
5.2	Verification Waveform Testbench Framework	17
5.3	Simulation Waveform Analysis	18
6	Conclusion	18

1 Introduction

1.1 Project Scope and Toolchain Paradigm

This document delineates the microarchitectural specification, structural design, and RTL implementation of a pipelined MIPS processing core using SystemVerilog Hardware Description Language (HDL). The design topology is thoroughly validated via behavioral and timing simulations within the ModelSim environment, followed by hardware synthesis, place-and-route (PnR), and static timing analysis (STA) utilizing Intel Quartus Prime to transform abstract register-transfer level source code into a physically realizable hardware implementation.

1.2 Architectural Optimization Strategy

The architectural analysis begins with a foundational, baseline single-cycle MIPS implementation to examine its operational limitations—specifically, its dependency on the worst-case critical path delay, which severely restricts the maximum operating clock frequency ($f_{\max}$). To address these performance bottlenecks, the architecture is systematically migrated toward a pipelined paradigm, introducing temporal parallelism to enhance instruction throughput and maximize processor performance. This step-by-step methodology provides a rigorous framework for evaluating how pipelining reshapes the fundamental MIPS datapath and control topology using modular SystemVerilog constructs.

1.3 Microarchitectural Challenges and Core Framework

Subsequent sections analyze the structural details of the pipelined datapath and its corresponding centralized control unit. Pipelining introduces microarchitectural conflicts, which are systematically resolved here:

- **Data Hazards:** Managed dynamically through localized operand bypass networks (Forwarding Unit) or selective bubble injection (Hardware Stalling).
- **Control Hazards:** Mitigated using early branch evaluation and pipeline flushes to maintain steady-state instruction execution.
- **Exception Management:** Implemented within the pipelined framework to handle synchronous interrupts, preserving overall processor state integrity.

1.4 Verification and Deliverable Overview

Ultimately, a comprehensive microarchitectural schematic of the complete MIPS processor is provided. The functional validity and hardware efficiency of this design are confirmed through verification wave traces from ModelSim alongside device utilization and timing analysis sheets compiled via Quartus Prime.

2 5-Stage Pipeline Architecture

The processing of any instruction within the MIPS core is partitioned into five distinct, sequential microarchitectural processing steps. Each stage is isolated from the next by intermediate boundary pipeline synchronization registers.

2.1 Microarchitectural Stage Description

1. **Instruction Fetch (IF):** Queries the 32-bit instruction word from the synchronous Instruction Memory using the current address stored in the Program Counter (PC). Simultaneously, increment logic computes the next sequential instruction address ($PC_{\text{new}} = PC + 4$).
2. **Instruction Decode / Register Fetch (ID):** Decodes the instruction word fields to extract opcodes, function codes, and register target addresses. The Control Unit maps these fields to establish control paths while the Register File fetches operand values from source indices (**rs** and **rt**). Simultaneously, sign-extension logic expands 16-bit immediate values to 32 bits.
3. **Execute / Address Calculation (EX):** Uses the Arithmetic Logic Unit (ALU) to evaluate logical computations, shifts, or arithmetic proofs. For memory-reference operations (**lw**, **sw**), the ALU computes the effective byte address by summing a base register with the sign-extended offset.
4. **Memory Access (MEM):** Interfaces with data storage profiles. When processing a load (**lw**) or store (**sw**) instruction, this stage reads or writes data within the Data Memory at the address computed by the ALU. Non-memory instructions route their results through this stage without modification.
5. **Write Back (WB):** Concludes the instruction life cycle by writing the computed result or retrieved memory data back into the Register File, using the destination register index (**rd** or **rt**) tracked through the pipeline stages.

2.2 Microarchitectural Layout Schematic

To visually inspect data routing, control signal propagation lines, and the interaction of interstage synchronization barriers across the execution core, the complete microarchitectural pipeline schematic is detailed below.

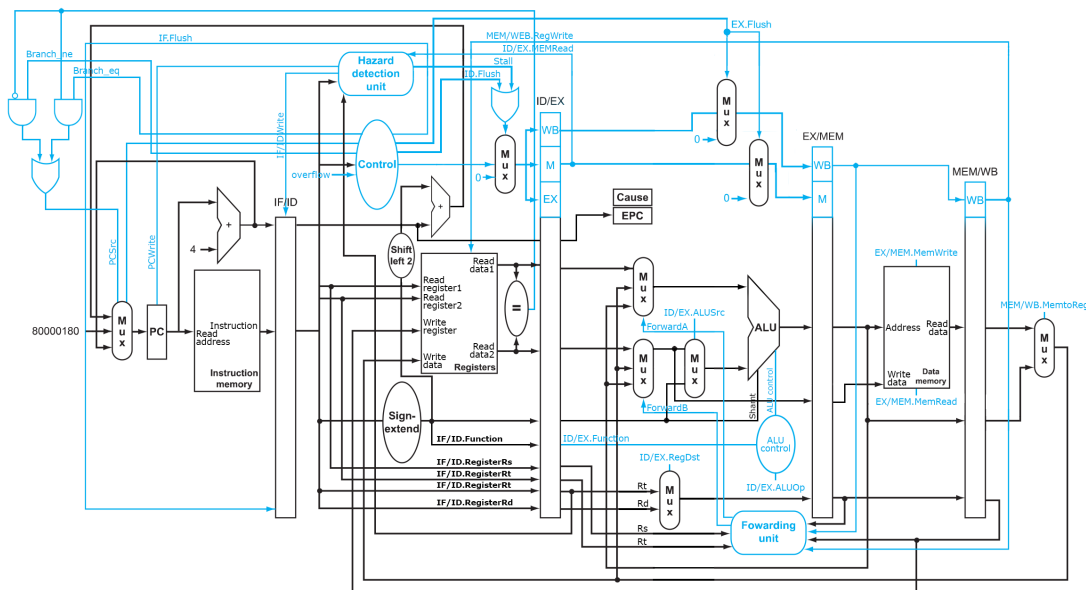


Figure 1: **Complete Integrated 5-Stage Pipelined MIPS Core Datapath Interconnect Diagram.**

The structural routing logic manages sequential dependencies across execution paths and provides a double-hazard override system to preserve microarchitectural state mapping using the following SystemVerilog implementation:

```

1 module Forwarding (
2     input          EX_MEM_RegWrite , MEM_WB_RegWrite ,
3     input          [4:0] EX_MEM_RegisterRd , MEM_WB_RegisterRd ,
4     ID_EX_RegisterRs , ID_EX_RegisterRt ,
5     output reg [1:0] ForwardA , ForwardB
6 );
7     always_comb begin : ForwardingA
8         // EX hazard (1) for operand path A
9         if (EX_MEM_RegWrite
10            && (EX_MEM_RegisterRd != 5'd0)
11            && (EX_MEM_RegisterRd == ID_EX_RegisterRs)) begin
12             ForwardA = 2'b10;
13         end
14         // MEM hazard (1) for operand path A (incorporating double-
15         hazard override)
16         else if (MEM_WB_RegWrite
17            && (MEM_WB_RegisterRd != 5'd0)
18            && (MEM_WB_RegisterRd == ID_EX_RegisterRs)) begin
19             ForwardA = 2'b01;
20         end
21         // No forwarding required (Default state)
22         else begin
23             ForwardA = 2'b00;
24         end
25     end
26     always_comb begin : ForwardingB
27         // EX hazard (2) for operand path B
28         if (EX_MEM_RegWrite
29            && (EX_MEM_RegisterRd != 5'd0)
30            && (EX_MEM_RegisterRd == ID_EX_RegisterRt)) begin
31             ForwardB = 2'b10;
32         end
33         // MEM hazard (2) for operand path B (incorporating double-
34         hazard override)
35         else if (MEM_WB_RegWrite
36            && (MEM_WB_RegisterRd != 5'd0)
37            && (MEM_WB_RegisterRd == ID_EX_RegisterRt)) begin
38             ForwardB = 2'b01;
39         end
40         // No forwarding required (Default state)
41         else begin
42             ForwardB = 2'b00;
43         end
44     end
45 endmodule : Forwarding

```

Listing 1: SystemVerilog Hardware Implementation of Data Forwarding Unit

3.2 Load-Use Data Hazards and Pipeline Stalling

Data forwarding cannot resolve dependencies when an instruction requires data from a load instruction (**lw**) immediately preceding it. Because the data is not retrieved until the end of the MEM stage, the dependent instruction must be stalled for one cycle.

When a stall condition is detected, the core freezes modifications to the **PC** and **IF/ID** registers for one cycle, while flushing the **ID/EX** control lines to zero. This injects a blank operation (bubble) into the execution path. The comprehensive SystemVerilog design for the integrated Hazard Detection Unit is structured as follows:

```

1 module Hazard_detection (
2     input          ID_EX_MemRead ,
3     input          [4:0] ID_EX_RegisterRt , IF_ID_RegisterRs ,
4     IF_ID_RegisterRt ,
5     output reg     PCWrite , IF_IDWrite , Stall
6 );
7     always_comb begin
8         if (ID_EX_MemRead &&
9             ((ID_EX_RegisterRt == IF_ID_RegisterRs) ||
10              (ID_EX_RegisterRt == IF_ID_RegisterRt))) begin
11             PCWrite      = 1'b0;
12             IF_IDWrite   = 1'b0;
13             Stall        = 1'b1;
14         end else begin
15             PCWrite      = 1'b1;
16             IF_IDWrite   = 1'b1;
17             Stall        = 1'b0;
18         end
19     end
20
21 endmodule : Hazard_detection

```

Listing 2: SystemVerilog Hardware Implementation of Hazard Detection Unit

3.3 Control Hazards (Branch Flushes)

Control hazards occur during conditional branch instructions (**beq**, **bne**). To reduce the penalty of mispredicted paths, this architecture evaluates branch targets and conditions early in the **ID** stage using equality comparators. If a branch condition is met, the invalid instruction currently in the **IF** stage is flushed by clearing the **IF/ID** register.

4 Hardware Component Implementation

This section aggregates the detailed register-transfer level (RTL) implementations of individual modules making up the pipelined processor architecture, ordered systematically from core datapath blocks to control managers.

4.1 Program Counter Register (PC)

```

1 module PC #(parameter WIDTH = 32)(
2     input clk, reset_n, PCWrite ,
3     input [WIDTH-1:0] PC_load ,
4     output reg [WIDTH-1:0] PC_next
5 );

```

```

6     always @(posedge clk) begin
7         if (~reset_n)
8             PC_next <= 0;
9         else if (PCWrite)
10            PC_next <= PC_load;
11     end
12 endmodule : PC

```

Listing 3: PC Register Source Code

4.2 Instruction Memory Storage Block

```

1 module InstructionMemory #(
2     parameter MEM_WIDTH = 8,
3     parameter WORD_WIDTH = 32,
4     parameter MEM_DEPTH = 1024,
5     parameter ADDR_SIZE = 32
6 ) (
7     input clk, IF_Flush, IF_IDWrite,
8     input [ADDR_SIZE-1:0] read_address,
9     output reg [WORD_WIDTH-1:0] instruction
10 );
11     reg [MEM_WIDTH-1:0] Instructions [0:MEM_DEPTH-1];
12
13     always @(posedge clk) begin
14         if (IF_Flush)
15             instruction <= 0;
16         else if (IF_IDWrite)
17             instruction <= {Instructions[read_address], Instructions[
18 read_address+1], Instructions[read_address+2], Instructions[
19 read_address+3]};
20     end
21 endmodule

```

Listing 4: Instruction Memory Module Source Code

4.3 Multi-Port Unified Register File

```

1 module RegisterFile#(
2     parameter REG_WIDTH = 32,
3     parameter REG_DEPTH = 32,
4     parameter ADDR_SIZE = 5
5 ) (
6     input clk, RegWrite,
7     input [ADDR_SIZE-1:0] read_register1, read_register2, write_register
8     ,
9     input [REG_WIDTH-1:0] write_data,
10    output [REG_WIDTH-1:0] read_data1, read_data2
11 );
12     reg [REG_WIDTH-1:0] Registers [0:REG_DEPTH-1];
13
14     assign read_data1 = Registers[read_register1];
15     assign read_data2 = Registers[read_register2];
16
17     always @(negedge clk) begin
18         if (RegWrite)

```



```

18     Registers[write_register] <= write_data;
19     end
20 endmodule

```

Listing 5: Register File Architecture Source Code

4.4 Arithmetic Logic Unit (ALU)

```

1 module ALU #(
2     parameter WIDTH = 32
3 ) (
4     input [3:0] ALU_control,
5     input [4:0] shamt,
6     input signed [WIDTH-1:0] A, B,
7     output logic signed [WIDTH-1:0] ALU_result,
8     output logic Zero, overflow
9 );
10 assign Zero = (ALU_result == 0)? 1'b1 : 1'b0;
11
12 always_comb begin
13     case(ALU_control)
14         4'b0000: begin //AND
15             ALU_result = A & B;
16             overflow = 0;
17         end
18         4'b0001: begin //OR
19             ALU_result = A | B;
20             overflow = 0;
21         end
22         4'b0010: begin //ADD
23             ALU_result = A + B;
24             if ((A[WIDTH-1] == B[WIDTH-1]) && (ALU_result[WIDTH-1]
25 != A[WIDTH-1]))
26                 overflow = 1;
27             else
28                 overflow = 0;
29         end
30         4'b0110: begin //SUB
31             ALU_result = A - B;
32             if ((A[WIDTH-1] != B[WIDTH-1]) && (ALU_result[WIDTH-1]
33 != A[WIDTH-1]))
34                 overflow = 1;
35             else
36                 overflow = 0;
37         end
38         4'b0111: begin //SLT
39             if(A < B)
40                 ALU_result = 1;
41             else
42                 ALU_result = 0;
43             overflow = 0;
44         end
45         4'b1100: begin //NOR
46             ALU_result = ~(A | B);
47             overflow = 0;
48         end
49         4'b1111: begin //SLL

```

```

48         ALU_result = B << shamt;
49         overflow = 0;
50     end
51     default: begin
52         ALU_result = A & B;
53         overflow = 0;
54     end
55 endcase
56 end
57 endmodule

```

Listing 6: ALU Computational Block Source Code

4.5 ALU Local Decoding Logic Controller

```

1 module ALUControl (
2     input [1:0] ALUOp,
3     input [5:0] funct,
4     output reg [3:0] ALU_control
5 );
6     always_comb begin
7         if(ALUOp == 2'b00)
8             ALU_control = 4'b0010; // lw/sw -> ADD
9         else if(ALUOp == 2'b01)
10            ALU_control = 4'b0110; // branch -> SUB
11        else begin
12            case(funct)
13                6'b100000: ALU_control = 4'b0010; // add
14                6'b100010: ALU_control = 4'b0110; // sub
15                6'b100100: ALU_control = 4'b0000; // and
16                6'b100101: ALU_control = 4'b0001; // or
17                6'b101010: ALU_control = 4'b0111; // slt
18                6'b000000: ALU_control = 4'b1111; // sll
19                default: ALU_control = 4'b0010; // default add
20            endcase
21        end
22    end
23 endmodule : ALUControl

```

Listing 7: ALU Control Decoder Source Code

4.6 Synchronous Byte-Addressable Data Memory

```

1 module DataMemory #(
2     parameter MEM_WIDTH = 8,
3     parameter WORD_WIDTH = 32,
4     parameter MEM_DEPTH = 1024,
5     parameter ADDR_SIZE = 32
6 ) (
7     input clk, MemRead, MemWrite,
8     input [ADDR_SIZE-1:0] address,
9     input [WORD_WIDTH-1:0] write_data,
10    output reg [WORD_WIDTH-1:0] read_data
11 );
12    reg [MEM_WIDTH-1:0] Data [0:MEM_DEPTH-1];
13

```

```

14     always @(posedge clk) begin
15         if(MemWrite) begin
16             Data[address]    <= write_data[31:24];
17             Data[address+1]  <= write_data[23:16];
18             Data[address+2]  <= write_data[15:8];
19             Data[address+3]  <= write_data[7:0];
20         end
21         if(MemRead)
22             read_data <= {Data[address], Data[address+1], Data[address
23 +2], Data[address+3]};
24     end
25 endmodule

```

Listing 8: Data Memory Block Source Code

4.7 Central Control Logic Array Unit

```

1 module Main_control (
2     input [5:0] opcode,
3     input overflow, PCSrc,
4     output RegDst, Branch_eq, Branch_ne, IF_Flush, MemtoReg, EX_Flush,
5     ID_Flush,
6     output reg MemRead, MemWrite, ALUSrc, RegWrite,
7     output reg [1:0] ALUOp,
8     output exception
9 );
10     wire invalid_opcode;
11
12     assign RegDst = ALUOp[1];
13     assign Branch_eq = (opcode == 5)? 1'b0 : ALUOp[0];
14     assign Branch_ne = (opcode == 5)? 1'b1 : 1'b0;
15     assign IF_Flush = PCSrc;
16     assign MemtoReg = (opcode == 8)? 1'b0 : ALUSrc;
17     assign invalid_opcode = ((opcode != 0) && (opcode != 35) && (opcode
18 != 43) && (opcode != 4) && (opcode != 5) && (opcode != 8))? 1'b1 : 1'
19 b0;
20     assign EX_Flush = overflow;
21     assign ID_Flush = (invalid_opcode)? 1'b1 : 1'b0;
22     assign exception = (invalid_opcode || overflow)? 1'b1 : 1'b0;
23
24     always_comb begin
25         case(opcode)
26             0: begin //R-format
27                 ALUOp = 2; ALUSrc = 0; MemRead = 0; MemWrite = 0;
28                 RegWrite = 1;
29             end
30             35: begin //lw
31                 ALUOp = 0; ALUSrc = 1; MemRead = 1; MemWrite = 0;
32                 RegWrite = 1;
33             end
34             43: begin //sw
35                 ALUOp = 0; ALUSrc = 1; MemRead = 0; MemWrite = 1;
36                 RegWrite = 0;
37             end
38             4: begin //beq
39                 ALUOp = 1; ALUSrc = 0; MemRead = 0; MemWrite = 0;
40                 RegWrite = 0;

```

```

34         end
35         5: begin //bne
36             ALUOp = 1; ALUSrc = 0; MemRead = 0; MemWrite = 0;
37         RegWrite = 0;
38         end
39         8: begin //addi
40             ALUOp = 0; ALUSrc = 1; MemRead = 0; MemWrite = 0;
41         RegWrite = 1;
42         end
43         default: begin
44             ALUOp = 2; ALUSrc = 0; MemRead = 0; MemWrite = 0;
45         RegWrite = 1;
46         end
47     endcase
48 end
49 endmodule : Main_control

```

Listing 9: Main Controller Logic Array Source Code

4.8 Top Intertwined Integrated Control Subsystem

```

1 module MIPS_control (
2     input [5:0] opcode,
3     input overflow, PCSrc,
4     output RegDst, Branch_eq, Branch_ne, IF_Flush, MemtoReg, EX_Flush,
5     ID_Flush,
6     output MemRead, MemWrite, ALUSrc, RegWrite,
7     output [1:0] ALUOp,
8     output exception,
9
10    input EX_MEM_RegWrite, MEM_WB_RegWrite,
11    input [4:0] EX_MEM_RegisterRd, MEM_WB_RegisterRd, ID_EX_RegisterRs,
12    ID_EX_RegisterRt,
13    output [1:0] ForwardA, ForwardB,
14
15    input ID_EX_MemRead,
16    input [4:0] IF_ID_RegisterRs, IF_ID_RegisterRt,
17    output PCWrite, IF_IDWrite, Stall
18 );
19
20 Main_control Main_Control_unit(opcode, overflow, PCSrc, RegDst,
21 Branch_eq, Branch_ne, IF_Flush, MemtoReg, EX_Flush, ID_Flush, MemRead,
22 MemWrite, ALUSrc, RegWrite, ALUOp, exception);
23 Forwarding Forwarding_unit(EX_MEM_RegWrite, MEM_WB_RegWrite,
24 EX_MEM_RegisterRd, MEM_WB_RegisterRd, ID_EX_RegisterRs,
25 ID_EX_RegisterRt, ForwardA, ForwardB);
26 Hazard_detection Hazard_detection_unit(ID_EX_MemRead,
27 ID_EX_RegisterRt, IF_ID_RegisterRs, IF_ID_RegisterRt, PCWrite,
28 IF_IDWrite, Stall);
29
30 endmodule : MIPS_control

```

Listing 10: MIPS Control Subsystem Encapsulation Block

4.9 Core Pipeline Structural Datapath Unit

```

1 module MIPS_datapath #(
2     parameter MEM_WIDTH = 8,
3     parameter WORD_WIDTH = 32,
4     parameter DEPTH = 1024,
5     parameter MEM_ADDR_SIZE = 32,
6     parameter REG_DEPTH = 32,
7     parameter REG_ADDR_SIZE = 5
8 ) (
9     input clk, reset_n,
10    output logic [5:0] IF_ID_Opcode,
11    output logic overflow, PCSrc,
12    input RegDst, Branch_eq, Branch_ne, IF_Flush, MemtoReg, EX_Flush,
    ID_Flush,
13    input MemRead, MemWrite, ALUSrc, RegWrite,
14    input [1:0] ALUOp,
15    input exception,
16
17    output logic EX_MEM_RegWrite, MEM_WB_RegWrite,
18    output logic [4:0] EX_MEM_RegisterRd, MEM_WB_RegisterRd,
    ID_EX_RegisterRs, ID_EX_RegisterRt,
19    input [1:0] ForwardA, ForwardB,
20
21    output logic ID_EX_MemRead,
22    output logic [4:0] IF_ID_RegisterRs, IF_ID_RegisterRt,
23    input PCWrite, IF_IDWrite, Stall,
24
25    output logic [WORD_WIDTH-1:0] RegWriteData, RegReadData1,
    RegReadData2,
26    output logic EX_MEM_MemRead, EX_MEM_MemWrite,
27    output logic [WORD_WIDTH-1:0] EX_MEM_ALU_result, EX_MEM_MemWriteData,
    MemReadData,
28    output logic [WORD_WIDTH-1:0] PC_out, instruction,
29    output logic [WORD_WIDTH-1:0] EPC
30 );
31
32 // Stage 1 Connections
33 logic equality;
34 logic [WORD_WIDTH-1:0] PC_in, IF_ID_next_Instruction, branch_target,
    next_Instruction;
35
36 assign PCSrc = ((Branch_eq & equality) || (Branch_ne & !equality))?
    1'b1 : 1'b0;
37 assign next_Instruction = PC_out + 4;
38
39 PC #(WORD_WIDTH) PC_inst(clk, reset_n, PCWrite, PC_in, PC_out);
40 InstructionMemory #(MEM_WIDTH, WORD_WIDTH, DEPTH, MEM_ADDR_SIZE)
    InstructionMemory_inst(clk, IF_Flush, IF_IDWrite, PC_out, instruction
    );
41
42 always_comb begin : PC_mux
43     if(exception) PC_in = 32'h80000180;
44     else if(PCSrc) PC_in = branch_target;
45     else PC_in = next_Instruction;
46 end
47
48 always @(posedge clk) begin : IF_ID_pipeline_register
49     if(IF_IDWrite) IF_ID_next_Instruction <= next_Instruction;

```

```

50     end
51
52     // Stage 2 Connections
53     logic ID_EX_RegDst, ID_EX_MemtoReg, ID_EX_MemWrite, ID_EX_ALUSrc,
ID_EX_RegWrite;
54     logic [1:0] ID_EX_ALUOp;
55     logic [4:0] IF_ID_RegisterRd, ID_EX_RegisterRd, IF_ID_Shamt,
ID_EX_Shamt;
56     logic [5:0] ID_EX_Funct, IF_ID_Funct;
57     logic [15:0] IF_ID_Immediate;
58     logic [WORD_WIDTH-1:0] IF_ID_SignedImmediate, ID_EX_RegReadData1,
ID_EX_RegReadData2, ID_EX_SignedImmediate, IF_ID_ShiftedImmediate,
ID_EX_next_Instruction;
59
60     assign IF_ID_Opcode = instruction[31:26];
61     assign IF_ID_RegisterRs = instruction[25:21];
62     assign IF_ID_RegisterRt = instruction[20:16];
63     assign IF_ID_RegisterRd = instruction[15:11];
64     assign IF_ID_Shamt = instruction[10:6];
65     assign IF_ID_Funct = instruction[5:0];
66     assign IF_ID_Immediate = instruction[15:0];
67     assign IF_ID_SignedImmediate = {{16{IF_ID_Immediate[15]}}},
IF_ID_Immediate};
68     assign IF_ID_ShiftedImmediate = {IF_ID_SignedImmediate[WORD_WIDTH
-3:0], {2{1'b0}}};
69     assign equality = (RegReadData1 == RegReadData2)? 1'b1 : 1'b0;
70     assign branch_target = IF_ID_ShiftedImmediate +
IF_ID_next_Instruction;
71
72     RegisterFile #(WORD_WIDTH, REG_DEPTH, REG_ADDR_SIZE)
RegisterFile_inst(clk, MEM_WB_RegWrite, IF_ID_RegisterRs,
IF_ID_RegisterRt, MEM_WB_RegisterRd, RegWriteData, RegReadData1,
RegReadData2);
73
74     always @(posedge clk) begin : ID_EX_pipeline_register
75         if(ID_Flush | Stall) begin
76             ID_EX_RegDst <= 0; ID_EX_MemtoReg <= 0; ID_EX_MemRead <= 0;
77             ID_EX_MemWrite <= 0; ID_EX_ALUSrc <= 0; ID_EX_RegWrite <= 0;
ID_EX_ALUOp <= 0;
78             end else begin
79                 ID_EX_RegDst <= RegDst; ID_EX_MemtoReg <= MemtoReg;
ID_EX_MemRead <= MemRead;
80                 ID_EX_MemWrite <= MemWrite; ID_EX_ALUSrc <= ALUSrc;
ID_EX_RegWrite <= RegWrite;
81                 ID_EX_ALUOp <= ALUOp; ID_EX_next_Instruction <=
IF_ID_next_Instruction;
82                 ID_EX_RegisterRs <= IF_ID_RegisterRs; ID_EX_RegisterRt <=
IF_ID_RegisterRt;
83                 ID_EX_RegisterRd <= IF_ID_RegisterRd; ID_EX_Funct <=
IF_ID_Funct;
84                 ID_EX_SignedImmediate <= IF_ID_SignedImmediate; ID_EX_Shamt
<= IF_ID_Shamt;
85                 ID_EX_RegReadData1 <= RegReadData1; ID_EX_RegReadData2 <=
RegReadData2;
86                 EPC <= ID_EX_next_Instruction;
87             end
88         end
89

```

```

90 // Stage 3 Connections
91 logic Zero, EX_MEM_MemtoReg;
92 logic [3:0] ALU_control;
93 logic [4:0] RegisterRd;
94 logic [WORD_WIDTH-1:0] A, B, B_ReadData, ALU_result;
95
96 assign A = (ForwardA == 2'b00) ? ID_EX_RegReadData1 : (ForwardA ==
2'b01) ? RegWriteData : EX_MEM_ALU_result;
97 assign B_ReadData = (ForwardB == 2'b00) ? ID_EX_RegReadData2 : (
ForwardB == 2'b01) ? RegWriteData : EX_MEM_ALU_result;
98 assign B = (ID_EX_ALUSrc == 1'b1)? ID_EX_SignedImmediate :
B_ReadData;
99 assign RegisterRd = (ID_EX_RegDst == 1'b1)? ID_EX_RegisterRd :
ID_EX_RegisterRt;
100
101 ALU #(WORD_WIDTH) ALU_inst(ALU_control, ID_EX_Shamt, A, B,
ALU_result, Zero, overflow);
102 ALUControl ALUControl_inst(ID_EX_ALUOp, ID_EX_Funct, ALU_control);
103
104 always @(posedge clk) begin : EX_MEM_pipeline_register
105     if(EX_Flush) begin
106         EX_MEM_MemtoReg <= 0; EX_MEM_MemRead <= 0; EX_MEM_MemWrite
<= 0; EX_MEM_RegWrite <= 0;
107     end else begin
108         EX_MEM_MemtoReg <= ID_EX_MemtoReg; EX_MEM_MemRead <=
ID_EX_MemRead;
109         EX_MEM_MemWrite <= ID_EX_MemWrite; EX_MEM_RegWrite <=
ID_EX_RegWrite;
110         EX_MEM_ALU_result <= ALU_result; EX_MEM_MemWriteData <=
B_ReadData;
111         EX_MEM_RegisterRd <= RegisterRd;
112     end
113 end
114
115 // Stage 4 Connections
116 logic MEM_WB_MemtoReg;
117 logic [WORD_WIDTH-1:0] MEM_WB_ALU_result;
118
119 DataMemory #(MEM_WIDTH, WORD_WIDTH, DEPTH, MEM_ADDR_SIZE)
DataMemory_inst(clk, EX_MEM_MemRead, EX_MEM_MemWrite,
EX_MEM_ALU_result, EX_MEM_MemWriteData, MemReadData);
120
121 always @(posedge clk) begin : MEM_WB_pipeline_register
122     MEM_WB_ALU_result <= EX_MEM_ALU_result;
123     MEM_WB_RegWrite <= EX_MEM_RegWrite;
124     MEM_WB_RegisterRd <= EX_MEM_RegisterRd;
125     MEM_WB_MemtoReg <= EX_MEM_MemtoReg;
126 end
127
128 // Stage 5 Connections
129 assign RegWriteData = (MEM_WB_MemtoReg == 1'b1)? MemReadData :
MEM_WB_ALU_result;
130
131 endmodule : MIPS_datapath

```

Listing 11: MIPS Pipeline Datapath Interconnect Implementation

4.10 Top-Level MIPS Enclosure Wrapper

```

1 module MIPS_top #(
2     parameter WORD_WIDTH = 32,
3     parameter DEPTH = 1024,
4     parameter MEM_ADDR_SIZE = 32,
5     parameter REG_DEPTH = 32,
6     parameter REG_ADDR_SIZE = 5
7 ) (
8     input clk, reset_n,
9     output logic MEM_WB_RegWrite,
10    output logic [4:0] IF_ID_RegisterRs, IF_ID_RegisterRt,
11    MEM_WB_RegisterRd,
12    output logic [WORD_WIDTH-1:0] RegWriteData, RegReadData1,
13    RegReadData2,
14    output logic EX_MEM_MemRead, EX_MEM_MemWrite,
15    output logic [WORD_WIDTH-1:0] EX_MEM_ALU_result, EX_MEM_MemWriteData
16    , MemReadData,
17    output logic [WORD_WIDTH-1:0] PC_out, instruction,
18    output logic IF_Flush, IF_IDWrite,
19    output logic [WORD_WIDTH-1:0] EPC
20 );
21
22 logic [5:0] opcode;
23 logic overflow, PCSrc;
24 logic RegDst, Branch_eq, Branch_ne, MemtoReg, EX_Flush, ID_Flush;
25 logic MemRead, MemWrite, ALUSrc, RegWrite;
26 logic [1:0] ALUOp;
27 logic exception;
28 logic EX_MEM_RegWrite;
29 logic [4:0] EX_MEM_RegisterRd, ID_EX_RegisterRs, ID_EX_RegisterRt;
30 logic [1:0] ForwardA, ForwardB;
31 logic ID_EX_MemRead;
32 logic PCWrite, Stall;
33
34 MIPS_datapath Datapath(
35     clk, reset_n, opcode, overflow, PCSrc, RegDst, Branch_eq,
36     Branch_ne, IF_Flush, MemtoReg, EX_Flush, ID_Flush,
37     MemRead, MemWrite, ALUSrc, RegWrite, ALUOp, exception,
38     EX_MEM_RegWrite, MEM_WB_RegWrite,
39     EX_MEM_RegisterRd, MEM_WB_RegisterRd, ID_EX_RegisterRs,
40     ID_EX_RegisterRt, ForwardA, ForwardB, ID_EX_MemRead,
41     IF_ID_RegisterRs, IF_ID_RegisterRt, PCWrite, IF_IDWrite, Stall,
42     RegWriteData, RegReadData1, RegReadData2,
43     EX_MEM_MemRead, EX_MEM_MemWrite, EX_MEM_ALU_result,
44     EX_MEM_MemWriteData, MemReadData, PC_out, instruction, EPC);
45
46 MIPS_control Control(
47     opcode, overflow, PCSrc, RegDst, Branch_eq, Branch_ne, IF_Flush,
48     MemtoReg, EX_Flush, ID_Flush,
49     MemRead, MemWrite, ALUSrc, RegWrite, ALUOp, exception,
50     EX_MEM_RegWrite, MEM_WB_RegWrite,
51     EX_MEM_RegisterRd, MEM_WB_RegisterRd, ID_EX_RegisterRs,
52     ID_EX_RegisterRt, ForwardA, ForwardB, ID_EX_MemRead,
53     IF_ID_RegisterRs, IF_ID_RegisterRt, PCWrite, IF_IDWrite, Stall);
54 endmodule : MIPS_top

```

Listing 12: Top Module Architectural Assembly Connection Mapping Layer

5 Verification and Simulation Results

To prove the structural efficiency and logical robustness of the structural SystemVerilog implementation under hazard constraints, verification testing was executed.

5.1 Verification Firmware

The following complete MIPS assembly verification sequence was targeted to cross-compile array indexing loops, causing heavy computational data dependencies and immediate branch hazard penalties.

Listing 13: MIPS System Verification Validation Firmware Instruction Set

```

1  # Pipeline Verification Initialization Sequence
2  addi $t0, $zero, 0x0020      # $t0 <= Base Data Reference Pointer (0
   x20080020)
3  addi $t1, $zero, 0x0013      # $t1 <= Array element [0] value (0
   x20090013)
4  addi $t2, $zero, 0x0095      # $t2 <= Array element [1] value (0
   x200A0095)
5  addi $t3, $zero, 0x0018      # $t3 <= Array element [2] value (0
   x200B0018)
6  addi $t4, $zero, 0x0003      # $t4 <= Array element [3] value (0
   x200C0003)
7  addi $t5, $zero, 0x0063      # $t5 <= Array element [4] value (0
   x200D0063)
8  addi $t6, $zero, 0x0047      # $t6 <= Array element [5] value (0
   x200E0047)
9  addi $t7, $zero, 0x0005      # $t7 <= Loop Index Counter (0
   x200F0005)
10 addi $s7, $zero, 0x0000      # $s7 <= Accumulator Clear (0
   x20170000)
11
12 # Memory Map Data Allocations
13 sw    $t1, 0x0000($t0)        # array[0] = 0x0013 (0
   xAD090000)
14 sw    $t2, 0x0004($t0)        # array[1] = 0x0095 (0
   xAD0A0004)
15 sw    $t3, 0x0008($t0)        # array[2] = 0x0018 (0
   xAD0B0008)
16 sw    $t4, 0x000C($t0)        # array[3] = 0x0003 (0
   xAD0C000C)
17 sw    $t5, 0x0010($t0)        # array[4] = 0x0063 (0
   xAD0D0010)
18 sw    $t6, 0x0014($t0)        # array[5] = 0x0047 (0
   xAD0E0014)
19
20 loop:
21 sll    $s1, $t7, 0x0002        # $s1 <= Index offset scaling (*4) (0
   x000F8880)
22 add    $s1, $s1, $t0            # $s1 <= Absolute target compute (0
   x02288820)
23 lw     $s2, 0x0000($s1)        # $s2 <= Load array[counter] (0
   x8E320000)
24 add    $s7, $s7, $s2            # $s7 <= Accumulate value (0
   x02F2B820)
25 addi   $t7, $t7, 0xFFFF        # $t7 <= Decrement index loop counter(-1) (0
   x21FFFFFF)

```

```

26 bne    $t7, $zero, 0xFFFFA      # If index != 0 branch back to loop (0
    x15E0FFFA)
27 sw     $s7, 0x0018($t0)          # Store final accumulation result (0
    xAD170018)

```

5.2 Verification Waveform Testbench Framework

```

1 module MIPS_tb ();
2     parameter WORD_WIDTH = 32;
3     parameter DEPTH = 1024;
4     parameter MEM_ADDR_SIZE = 32;
5     parameter REG_DEPTH = 32;
6     parameter REG_ADDR_SIZE = 5;
7
8     logic clk, reset_n;
9     logic MEM_WB_RegWrite;
10    logic [4:0] IF_ID_RegisterRs, IF_ID_RegisterRt, MEM_WB_RegisterRd;
11    logic [WORD_WIDTH-1:0] RegWriteData, RegReadData1, RegReadData2;
12    logic EX_MEM_MemRead, EX_MEM_MemWrite;
13    logic [WORD_WIDTH-1:0] EX_MEM_ALU_result, EX_MEM_MemWriteData,
    MemReadData;
14    logic [WORD_WIDTH-1:0] PC_out, instruction;
15    logic IF_Flush, IF_IDWrite;
16    logic [WORD_WIDTH-1:0] EPC;
17
18    MIPS_top DUT(clk, reset_n, MEM_WB_RegWrite, IF_ID_RegisterRs,
    IF_ID_RegisterRt, MEM_WB_RegisterRd, RegWriteData, RegReadData1,
19    RegReadData2, EX_MEM_MemRead, EX_MEM_MemWrite, EX_MEM_ALU_result
    , EX_MEM_MemWriteData, MemReadData, PC_out, instruction,
20    IF_Flush, IF_IDWrite, EPC);
21
22    initial begin
23        clk = 0;
24        forever #1 clk = ~clk;
25    end
26
27    initial begin
28        $readmemh("../Memory_Initialization/InstructionMemory.dat", DUT.
    Datapath.InstructionMemory_inst.Instructions);
29        $readmemh("../Memory_Initialization/DataMemory.dat", DUT.
    Datapath.DataMemory_inst.Data);
30        $readmemh("../Memory_Initialization/RegisterFile.dat", DUT.
    Datapath.RegisterFile_inst.Registers);
31
32        reset_n = 0;
33        @(negedge clk) reset_n = 1;
34
35        repeat(75) @(negedge clk);
36        $stop;
37    end
38 endmodule : MIPS_tb

```

Listing 14: System Verification Simulation Testbench Environment Source Code

5.3 Simulation Waveform Analysis

To validate the real-time operational execution of the processing engine, the timing wave transitions generated from the verification testbench are plotted below.

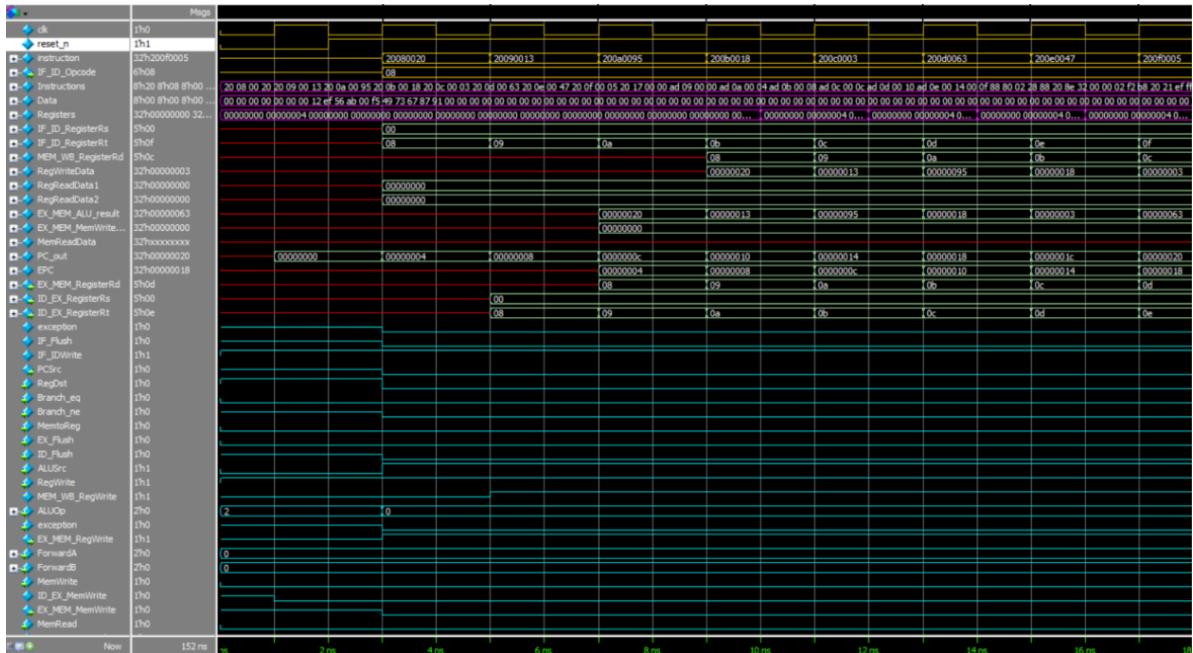


Figure 3: QuestaSim Behavioral Simulation Waveform Trace Delineating Instruction Execution Paths, Pipeline Hazard Recovery, and Accumulator Register Updates.

The simulation waveforms verified that the Hazard Detection Unit correctly identified the data dependency between the `lw $t1` and `or $t2` instructions. It correctly injected a single-cycle bubble during Clock Cycle 6 (CC6) and maintained structural consistency without corrupting downstream registers.

6 Conclusion

This 5-stage pipelined MIPS processor project demonstrates a fully functional microarchitecture designed to address structural, data, and control challenges. By implementing specialized hardware mechanisms like operand bypassing networks (Forwarding Unit) and hazard stalling control structures, data hazards and load-use constraints were resolved automatically. The resulting core maintains a stable instruction pipeline that meets the theoretical requirements of the *Computer Organization* course under the guidance of Dr. Hatem Zakaria at Benha University.